

Central Hub

Technical Specification

Version 0.1 Draft
April 2026
For Internal Review

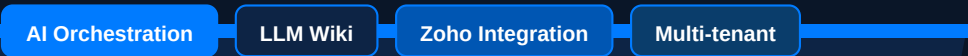


Table of Contents

- 1. Executive Summary**
- 2. Vision & Strategic Goals**
- 3. System Architecture Overview**
 - 3.1 High-Level System Map
 - 3.2 Core Task Lifecycle Flow
 - 3.3 Daily Digest Flow
- 4. Core Design Principles**
- 5. Knowledge Base System — LLM Wiki Approach**
 - 5.1 Two-Tier Knowledge Structure
 - 5.2 KB Ownership
 - 5.3 Wiki Lint Job
- 6. AI Orchestration Layer**
 - 6.1 Classification Engine
 - 6.2 Requirements Assessment
 - 6.3 Plan Generation
 - 6.4 Execution Engine
 - 6.5 Playbook Library
- 7. Feature Modules**
 - 7A Onboarding & Client Hub
 - 7B Project Management & Zoho
 - 7C Developer Workflow
 - 7D Daily Digest Engine
 - 7E Reply Generation
- 8. Data Model**
- 9. LLM Model Selection & Observability**
- 10. Integration Map**
- 11. MVP Scope & Acceptance Criteria**
- 12. Phase Roadmap**
- 13. Success Metrics**
- 14. Decision Log**
- 15. Open Items**

1. Executive Summary

The WebriQ Central Hub is an internal operations platform serving three connected functions:

Operational Dashboard	Unified daily work surface for PMs and Developers replacing manual Zoho navigation.
AI Orchestration Layer	Intelligent pipeline that classifies tasks, validates requirements, generates plans, and executes lightweight work with human approval gates.
Institutional Memory Store	Self-maintaining LLM Wiki knowledge base accumulating customer context, product knowledge, playbooks, and code documentation — improving every LLM action over time.

The Hub does not replace Zoho. It sits **above** it — synthesizing data from Zoho Projects, Zoho Desk, Sanity, GitHub, and product APIs into a single intelligent operational layer.

2. Vision & Strategic Goals

Primary Goals

ID	Goal	Description
G1	LLM-Visible Operations	Every task/ticket immediately visible to and understood by the LLM with structured queryable context.
G2	Autonomous Lightweight Execution	For repeatable task types, the LLM executes end-to-end via Sanity API or GitHub with human approval gates.
G3	Continuous Learning	Every approved plan, rejection, resolved ticket, and edited reply feeds back into the KB and calibrates future LLM behavior.
G4	Zero Zoho Dependency for Daily Awareness	PMs and Devs start the day from the Hub digest. Zoho is system of record for execution only.
G5	Configurable Human-in-the-Loop	All automation gates controllable. At MVP everything is manually triggered. Autonomy unlocked incrementally.

What the Hub Is Not

- Not a replacement for Zoho Projects or Zoho Desk
- Not a customer-facing portal (Phase 1)
- Not a time tracking tool — Zoho remains the source of truth for hours
- Not a code editor or IDE

3. System Architecture Overview

3.1 High-Level System Map

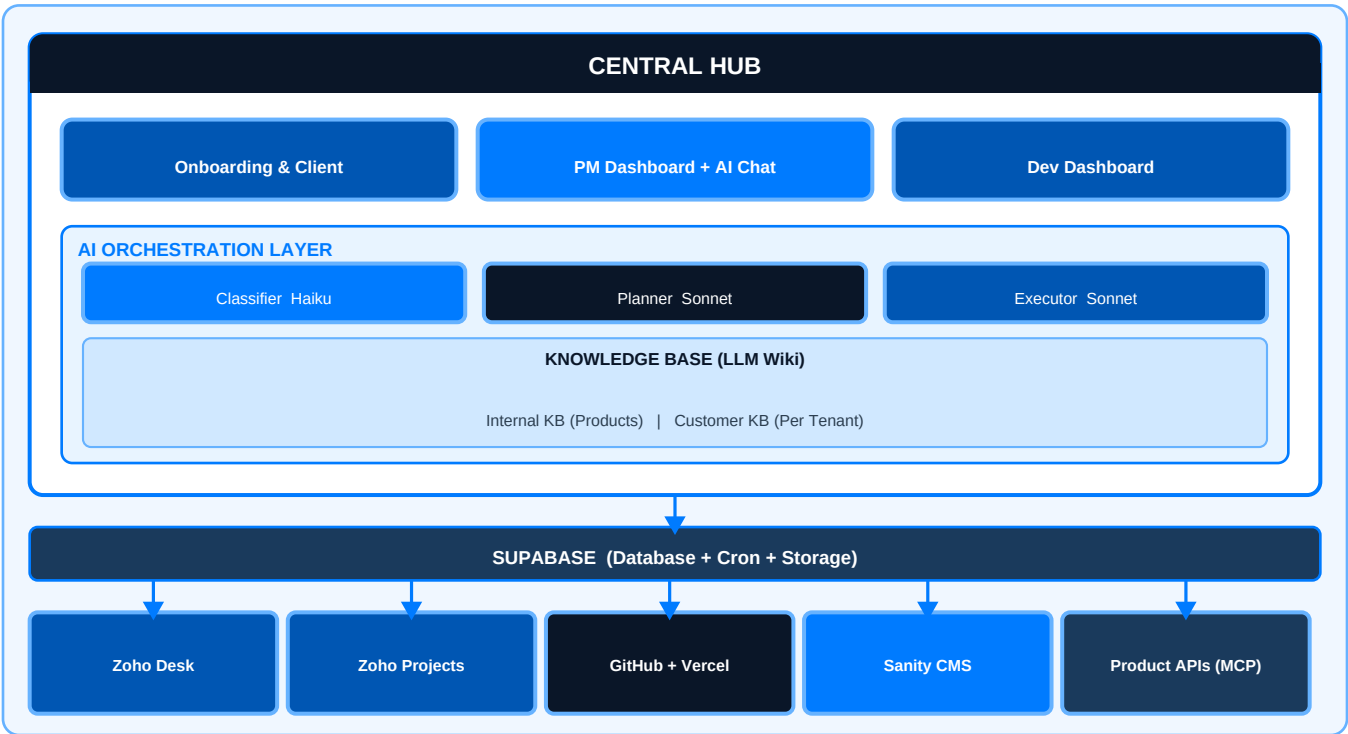


Figure 1: Central Hub sits above Zoho, GitHub, Sanity, and product APIs — synthesizing data through the AI Orchestration Layer backed by Supabase.

3.2 Core Task Lifecycle Flow

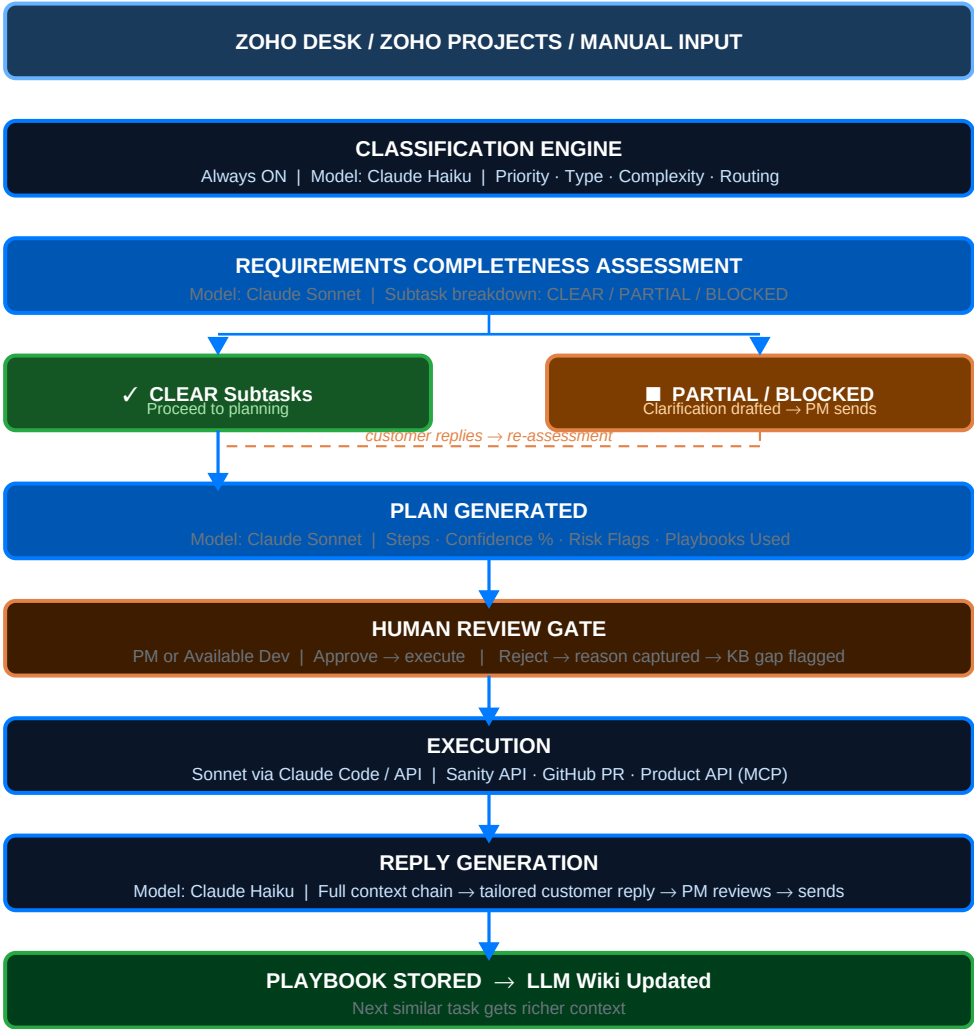


Figure 2: Every task flows from classification (always-on) through requirements assessment, plan generation, human approval, execution, and reply generation — with learning fed back into the Playbook Library.

3.3 Daily Digest Flow

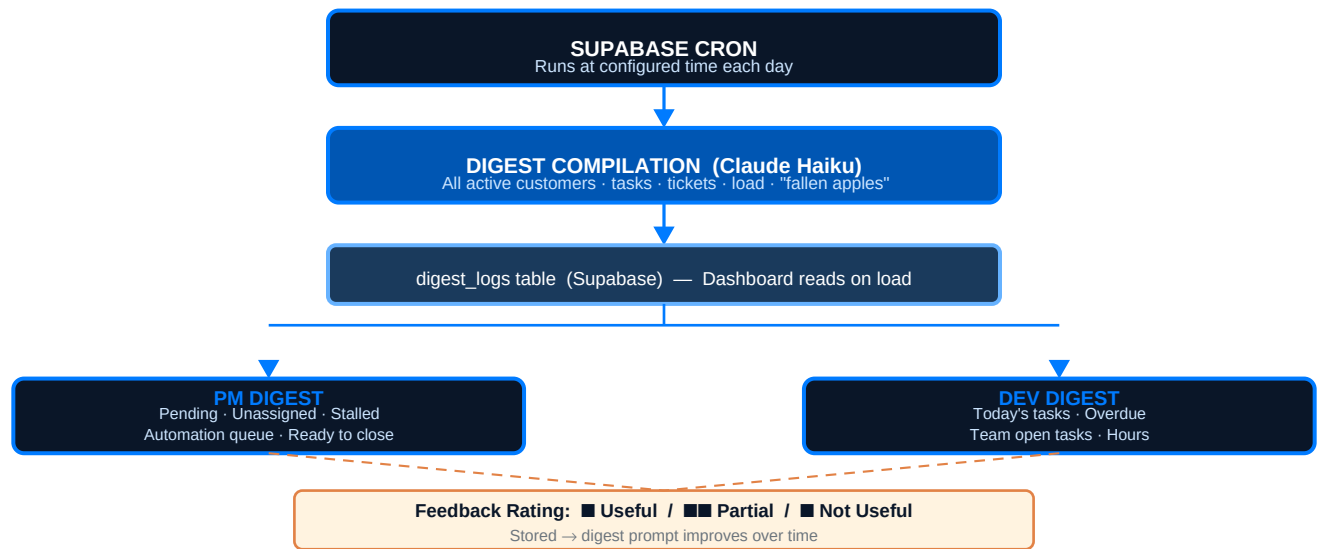


Figure 3: Supabase Cron compiles a pre-built digest daily. PM and Dev dashboards read on load — no live LLM calls. Feedback ratings improve the digest prompt over time.

4. Core Design Principles

ID	Principle	Description
P1	customer_id is the Universal Key	Every object in every system must reference customer_id. Created in Hub. Never duplicated.
P2	Classification Always On	Runs on every incoming task/ticket without exception or manual trigger.
P3	Requirements Before Execution	No plan generated for a task with incomplete requirements. A delayed clear start beats a fast vague one.
P4	Hub = Intent, Zoho = Execution	Status and intent flow Hub → Zoho. Direct Zoho edits flagged as DIRECT_ZOHO_EDIT.
P5	No Autonomous Production Writes	All code changes go to a feature branch. API actions log pre_state for reversibility. Human merges PRs.
P6	Context Chain is Sacred	Every task carries its full history: description, classification, assessment, clarification, plan, approval, execution, reply.
P7	Model Selection is Configurable	Haiku for high-frequency tasks. Sonnet for reasoning. Configurable per layer via DB without code changes.
P8	Manual First, Autonomous Later	All gates require human initiation at MVP. Autonomy unlocked incrementally as playbook confidence justifies it.

5. Knowledge Base System — LLM Wiki Approach

Overview

The Hub's knowledge base uses Karpathy's LLM Wiki approach rather than traditional RAG. The LLM is simultaneously the compiler, librarian, and query engine. Humans curate what goes in. The LLM does everything else.

Traditional RAG	LLM Wiki (Karpathy Approach)
Raw documents stored, retrieved at query time	LLM compiles documents into structured wiki at ingest
LLM rediscovers knowledge from scratch every query	Knowledge accumulated, cross-linked, and grows over time
No accumulation between sessions	Every interaction enriches the wiki permanently

5.1 Two-Tier Knowledge Structure

Directory	Owner	Contents
internal/raw/products/*	WebriQ Dev team	Product how-tos, architecture docs per product line
internal/raw/shared-schemas/	Dev team	Shared Sanity schemas, CMS config patterns
internal/wiki/	System (LLM auto)	LLM-compiled concept pages, how-tos, index
customers/{id}/raw/pm-docs/	PM per account	Specs, briefs, brand guides, stakeholder contacts
customers/{id}/raw/code-structure/	Dev per account	Code structure MDs, repo guides, technical patterns
customers/{id}/raw/tickets-resolved/	System (auto)	Closed ticket threads for pattern learning
customers/{id}/wiki/	System (LLM auto)	Per-customer compiled context, patterns, playbooks

5.2 Knowledge Base Ownership

Layer	Owner	Update Trigger
Internal Product KB	WebriQ team	Manual upload or product release
Customer KB — Documents	PM per account	PM upload anytime
Customer KB — Code Wiki	Dev per account	Dev upload or post-execution
Customer KB — Playbooks	System (auto)	On every completed task
Customer KB — Resolved Tickets	System (auto)	On every ticket close

5.3 Wiki Lint Job (Versioning & Health)

A Supabase Cron job runs **weekly** — LLM audits each wiki for contradictions, stale product references, orphan pages, and missing concept stubs.

- Contradictions between pages → flagged for PM review
- Content referencing deprecated product version → flagged STALE
- Orphan pages with no backlinks → flagged for archival
- Playbooks referencing deleted API endpoints → flagged INVALID

6. AI Orchestration Layer

6.1 Classification Engine

Model: Claude Haiku | **Trigger:** Every incoming task/ticket (webhook or manual) | **Always ON — no toggle**

Task Type Taxonomy — Phase 1

Task Type	LLM Eligible	Execution Method	Min Requirements
CONTENT_UPDATE	YES	Sanity API	Content file, target page, publish date
SETTINGS_CHANGE	YES	Product API (MCP)	Setting name, new value, target product
BLOG_PUBLISH	YES	Sanity API	Content file, metadata, publish schedule
ASSET_UPLOAD	YES	Sanity API / Storage	Asset files, target location, alt text
CODE_CHANGE_MINOR	YES (Sonnet)	GitHub PR	Scope, affected files, acceptance criteria
SEO_UPDATE	YES	Sanity API	Target page, new meta values
BUG_REPORT	NO (classify only)	Human Dev	Reproduction steps, expected vs actual
FEATURE_REQUEST	HUMAN_ONLY	Human Dev + PM	Full spec required
STRATEGIC	HUMAN_ONLY	PM only	Never enters automation pipeline

6.2 Requirements Completeness Assessment

Model: Claude Sonnet | **Trigger:** Immediately after classification for LLM-eligible tasks

■ *Critical: This is a first-class Supabase data record — not a prompt instruction. Every field is queryable. The entire downstream pipeline depends on the integrity of this record.*

Status	Meaning	Action
■ CLEAR	All required inputs present	Proceed to plan generation immediately
■ ■ PARTIAL	Some inputs missing	Flag generated, clarification drafted for PM
■ BLOCKED	Depends on unresolved subtask	Wait; downstream subtasks paused

Parallel Track: Subtasks that are CLEAR proceed to planning immediately. Blocked subtasks wait for clarification. Clear work does not stop while unclear work is resolved.

6.3 Plan Generation

Model: Claude Sonnet | **Trigger:** Manual ("Generate Plan" button) — or automatic when Automation Toggle is ON

Rejection Reason	Follow-up Action
PLAN_INCOMPLETE — missed steps	Re-generate with updated KB context
WRONG_APPROACH — right goal wrong method	Re-generate with corrected instructions
SCOPE_EXCEEDED — LLM tried to do too much	Re-generate with narrowed scope
KNOWLEDGE_GAP — lacked product context	Auto-create KB stub → PM fills → retry
MISCLASSIFICATION — wrong task type	Re-classification triggered → new plan

6.4 Execution Engine

Model: Claude Sonnet (via Claude Code / API) | **Trigger:** Manual — PM or Dev clicks "Execute" on approved plan

Mode	Where Work Happens	Output
Sanity API	Sanity API (per sanity_project_id)	Published or drafted content
GitHub PR	Feature branch on customer repo	PR link + Vercel/Netlify preview URL
Product API	Product MCP/Tool (provisioned access)	Settings updated

Rollback & Safety

- All code changes live on a feature branch — nothing touches production without human merge.
- API actions log pre_state for every mutation. A "Revert" button replays the inverse.
- PARTIAL_EXECUTION state: flagged for manual review, never auto-retried.
- Circuit breaker: 3 consecutive plan rejections/failures → automation auto-paused per customer.

6.5 Playbook Library

Every successfully completed task generates a playbook entry. When a new task arrives, the planner retrieves the top 3 most similar playbooks and injects them as context. This is the core learning mechanism — context gets richer over time without model fine-tuning.

Playbook Field	Description
original_task_description	What the customer originally asked for
classification_applied	Full classification record reference
implementation_plan	The plan that was approved and executed
execution_outcome	What happened, what was produced
embedding_summary	LLM-generated summary for semantic retrieval

Playbook Field	Description
status	ACTIVE STALE ARCHIVED (managed by lint job)

7. Feature Modules

7A — Onboarding & Client Information Hub

Each customer receives a unique, secure, login-free onboarding URL: `/onboard/{customer_id}`. Revisitable at any time. Progress saved automatically.

Section	StackShift	PublishForge	CiteForge	PipelineForge
A. Company Info	■	■	■	■
B. Contacts / Stakeholders	■	■	■	■
C. Project Goals	■	■	■	■
D. Content / Assets	■	■	—	—
E. Technical Requirements	■	—	■	■
F. Knowledgebase Materials	—	■	■	■

Customer Profile Key Fields

- `customer_id` — Universal key across all systems (never duplicated)
- `product_instances[]` — `product`, `instance_link`, `sanity_project_id`, `zoho_project_id`, etc.
- `automation_toggle` — Controls plan generation auto-trigger per customer
- `dedicated_developers[]` — Future tasks auto-routed to these developers
- `communication_tone` — formal | casual | technical (used in reply generation)
- `llm_excluded` — Human-only flag for sensitive clients or task types
- `onboarding_status` — `completion_pct`, `missing_fields`, `last_updated`

7B — Project Management & Zoho Integration

Zoho Event	Hub Action
Task status → COMPLETED	Surface in "Ready for Checking" queue. Notify PM.
Ticket status → RESOLVED / FOR REVIEW	Surface in PM checking queue.
Time log added	Update hours display in Hub (read-only).
Assignment changed (direct Zoho)	Update Hub, flag <code>DIRECT_ZOHO_EDIT</code> .
Task overdue	Add to digest overdue list. Notify PM.

7C — Developer Workflow

Intentionally lightweight. Hub is a daily reminder and quick access panel. All detailed work (comments, time logging) happens in Zoho.

Dashboard Element	Description
Today's Tasks	Assigned tasks with priority and direct Zoho link
Today's Tickets	Assigned tickets with priority and direct Zoho link
Overdue Items	Past-due items highlighted prominently
Team Open Tasks	Unassigned tasks any dev can self-assign. PM notified on assignment.
Hours This Week	Pulled from Zoho Projects (read-only)

7D — Daily Digest Engine

Model: Claude Haiku | Digest pre-compiled by Cron, stored in DB, dashboard reads on load. Not a live LLM call — keeps dashboard fast and costs predictable.

■ *Digest feedback: After each digest a one-tap rating (Useful / Partial / Not Useful) is stored. After 4 weeks, the digest prompt is reviewed and updated.*

7E — Reply Generation

Model: Claude Haiku | Trigger: Task reaches COMPLETED | Mode: Manual — PM reviews, edits if needed, sends. Full context chain produces a specific credible reply.

■ *Improvement loop: PM edits before sending are stored as diffs. Common patterns feed back into the reply generation system prompt as style guidance.*

8. Data Model — Core Tables

Table: customers

Column	Type	Notes
customer_id	UUID PK	Universal key across all systems
automation_toggle	BOOLEAN	Controls plan generation auto-trigger
llm_excluded	BOOLEAN	Excludes from all automation pipelines
communication_tone	TEXT	formal casual technical
onboarding_status	JSONB	completion_pct, missing_fields, last_updated

Table: customer_products

Column	Type	Notes
customer_id	UUID FK	References customers
product	TEXT	stackshift publishforge citeforge pipelineforge
sanity_project_id	TEXT	Per-tenant Sanity isolation key
zoho_project_id	TEXT	Linked Zoho project reference
dedicated_developers	UUID[]	Auto-assign future tasks to these devs

Table: classification_records

Column	Type	Notes
priority	TEXT	CRITICAL HIGH NORMAL LOW
task_type	TEXT	See Task Type Taxonomy
llm_eligible	TEXT	YES NO HUMAN_ONLY
confidence_score	INT	0–100
model_used, tokens_in, tokens_out	TEXT / INT	Observability and cost tracking

Table: requirements_assessments

Column	Type	Notes
overall_status	TEXT	COMPLETE PARTIAL BLOCKED
subtasks	JSONB	Per-subtask: status, missing_inputs, blocking IDs
clarification_draft	JSONB	Generated text + structured missing items

Column	Type	Notes
confidence_to_proceed	INT	0–100

Table: implementation_plans

Column	Type	Notes
steps	JSONB	step_number, description, action_type, target
confidence_score	INT	0–100
risk_flags	TEXT[]	LLM-identified risk areas
playbooks_used	UUID[]	KB references used to generate this plan
status	TEXT	PENDING_APPROVAL APPROVED REJECTED EXECUTING COMPLETE FAILED
approved_by / rejected_by	UUID + TEXT	Who actioned the review gate + rejection reason

Table: execution_records

Column	Type	Notes
outcome	TEXT	SUCCESS PARTIAL FAILED
outputs	JSONB	github_pr_url, preview_url, sanity_document_id, api_actions_log
pre_action_states	JSONB	State before each mutation — used for rollback
what_was_done / what_was_skipped	TEXT	LLM-generated execution summary

Table: playbooks

Column	Type	Notes
embedding_summary	TEXT	LLM summary used for semantic retrieval
status	TEXT	ACTIVE STALE ARCHIVED
last_validated	DATE	Set and updated by weekly lint job

Table: llm_invocation_logs

Column	Type	Notes
model_used, tokens_in, tokens_out	TEXT / INT	Per-call cost tracking
latency_ms	INT	Performance monitoring
orchestration_layer	TEXT	CLASSIFIER PLANNER EXECUTOR DIGEST REPLY
customer_id	UUID	Cost attribution per customer

9. LLM Model Selection & Observability

Orchestration Layer	Default Model	Rationale
Classification	Claude Haiku	High frequency, structured output, cost-sensitive
Daily Digest	Claude Haiku	Aggregation over known structured data
Reply Generation	Claude Haiku	Template-driven, high volume, customer-facing
Requirements Assessment	Claude Sonnet	Cross-KB reasoning, nuanced gap detection
Plan Generation	Claude Sonnet	Multi-step reasoning, subtask decomposition
Code Execution	Claude Sonnet	Complex implementation context
Orchestration / Routing	Claude Sonnet	Full context awareness required

■ *Model selection stored in a llm_config table — configurable per layer without code changes. Every invocation logs model, token counts, latency, customer_id. Soft cap per-customer daily token budget with PM alert.*

10. Integration Map

Zoho Desk

Direction	What	How
Hub → Zoho	Ticket assignment, reassignment, status update	Zoho Desk API
Zoho → Hub	Ticket status change, new ticket created	Webhook → Classification trigger

Zoho Projects

Direction	What	How
Hub → Zoho	Create project (onboarding), tasks, assignments, status	Zoho Projects API
Zoho → Hub	Task status change, time log added	Webhook
Zoho → Hub	Assignment changed (direct Zoho)	Webhook → flag DIRECT_ZOHO_EDIT

Sanity CMS

Direction	What	How
Hub → Sanity	Create/update document, publish, upload asset, schedule	Sanity API (per sanity_project_id)

GitHub

Direction	What	How
Hub → GitHub	Create feature branch, commit changes, open PR	GitHub API / Claude Code session
GitHub → Hub	PR status change	GitHub Webhook

Vercel / Netlify

Direction	What	How
V/N → Hub	Preview URL on branch deploy	Webhook

Zoho Cliq

Direction	What	How
Hub → Cliq	PM digest, Dev digest, plan approval, execution complete	Cliq API

11. MVP Scope & Acceptance Criteria

The MVP validates the core loop: task arrives → classified → requirements checked → plan generated → human approves → execution runs → reply generated.

M1 — Customer & Onboarding

- ■ Customer creation with customer_id
- ■ Dynamic onboarding form (modular, conditional logic)
- ■ Progressive completion (save, resume, share link)
- ■ File/asset upload support
- ■ PM dashboard: completion % and missing fields
- ■ Customer profile with product instance mapping

M2 — Classification Engine

- ■ Webhook listener for Zoho Desk and Zoho Projects
- ■ Classification runs on every incoming task/ticket
- ■ Classification record stored in Supabase with all fields
- ■ Low-confidence classifications surfaced for PM review
- ■ Manual re-classification available from Hub UI

M3 — Requirements Assessment

- ■ Subtask breakdown with CLEAR / PARTIAL / BLOCKED status
- ■ Clarification draft generated for PM review and send
- ■ Task flagged CLARIFICATION_NEEDED in digest when blocked
- ■ Re-assessment triggers when customer replies

M4 — Daily Digest

- ■ Supabase Cron running at configured time
- ■ PM digest: pending, unassigned, stalled, ready to close
- ■ Dev digest: assigned tasks, overdue, team unassigned
- ■ Digest stored in digest_logs — dashboard reads on load
- ■ Digest feedback rating (Useful / Partial / Not Useful)

M5 — Plan Generation (Manual)

- ■ "Generate Plan" button on any LLM-eligible task
- ■ Plan shown: steps, affected files, confidence score, risk flags
- ■ Approve / Reject with structured rejection reason
- ■ Rejection reasons linked to KB gap stubs

M6 — Execution: Content Updates

- ■ Approved plan triggers execution via Sanity API
- ■ Execution record stored with pre-action state for rollback
- ■ GitHub PR generated for code-type tasks
- ■ Vercel/Netlify preview URL captured and displayed

M7 — Zoho Sync

- ■ Hub creates Zoho project on customer onboarding
- ■ Hub pushes tasks and assignments to Zoho
- ■ Zoho status changes sync back via webhook
- ■ One-click direct task/ticket links from Hub to Zoho

M8 — Reply Generation

- ■ Reply draft generated on task COMPLETED
- ■ Draft shown in Hub for PM review and edit
- ■ Edit diff stored for prompt improvement loop

M9 — Developer Dashboard

- ■ Today's assigned tasks and tickets with direct Zoho links
- ■ Team unassigned tasks (self-assignable, PM notified)
- ■ Hours logged this week (pulled from Zoho)

M10 — LLM Wiki Knowledge Base (Seed)

- ■ Internal KB directory structure established
- ■ Seed playbooks authored for Content Update and Settings types
- ■ Customer KB scaffolded — PM and Dev upload working
- ■ Wiki lint cron scheduled weekly
- ■ LLM invocation logging active for all calls

MVP Acceptance Criteria

- **AC1.** A PM can onboard a new customer end-to-end without opening Zoho.
- **AC2.** A new Zoho Desk ticket appears in the Hub classified within 60 seconds.
- **AC3.** A Content Update task completes the full loop (classify → plan → execute → reply) without PM touching Zoho.
- **AC4.** A PM starts the day from the digest with full situational awareness without opening Zoho.
- **AC5.** A Developer can see assigned work and self-assign an available task from the Hub.

12. Phase Roadmap

Phase 0 — Infrastructure

Pre-MVP · No user-facing features · Foundation only

- Supabase schema deployed with all future fields stubbed (no Phase 2 migration needed)
- Zoho webhook infrastructure tested end-to-end with dummy data
- LLM Wiki directory structure in Supabase Storage established
- Model config table with Haiku/Sonnet assignments per layer
- LLM invocation logging and cost attribution from day one
- Seed content authored for Internal KB (product how-tos, shared Sanity schemas)
- Vercel AI SDK integrated (Next.js, chat interface, MCP support)
- Exit: Classification receives a Zoho webhook and produces a structured record.

Phase 1 — MVP

Core operational loop validated on real workloads

- All M1–M10 features from Section 11
- Exit: All 5 AC criteria met. Team using Hub as primary daily interface for 2+ weeks.

Phase 2 — Assisted Planning

Automation toggle enabled · Broader task type support

- Automation toggle per customer (plan auto-triggers when ON)
- Expanded task types: Settings, SEO Update, Asset Upload
- Playbook library actively accumulating from real executions
- Rejection reason data feeding KB gap identification workflow
- Per-customer circuit breaker (3 failures → auto-pause)
- Basic reporting dashboard (accuracy, approval rate, reply edit rate)
- Exit: 20+ executions in playbook library. Classification accuracy > 80%.

Phase 3 — Supervised Autonomy

Round-robin · Multi-dev · Learning flywheel measurable

- Round-robin auto-assignment with developer pool configuration
- Multiple developer assignment per task
- Workload-aware assignment with PM-configurable load thresholds
- Confidence calibration reporting (predicted vs actual outcomes)
- Wiki health dashboard (stale content, lint findings, KB coverage)
- Knowledge gap closure workflow (rejection → KB stub → PM fills → re-test)
- Exit: Confidence measurably improving. Digest rating > 75% Useful.

Phase 4 — Autonomous Operations

Trusted tiers · Auto-reply · Fine-tuning data pipeline

- Full autonomous execution for Tier 3 customers (PM notified post-execution)
- Auto-send reply with configurable review window
- LLM Wiki fine-tuning data export (structured for future model training)
- Cross-customer pattern recognition from aggregate execution data
- PM Handoff briefing: one-click project briefing generation for new PMs
- Exit: One customer running at full autonomous tier, 0 failures over 30 days.

13. Success Metrics

All metrics derivable from data already logged in Supabase. No separate analytics infrastructure required.

Metric	What it Measures	Phase 1 Target
Time from ticket arrival to assignment	Is the Hub reducing triage overhead?	Baseline
Classification accuracy rate	PM-rated: correct / total	> 75%
Plan approval rate (first submission)	Is requirements validation reducing rework?	> 60%
Rejection reason distribution	Where are knowledge / context gaps?	Logged weekly
Reply generation edit rate	How often PMs edit LLM-generated replies?	Baseline
Digest usefulness rating	Is the digest surfacing the right signals?	> 70% Useful
Tasks self-assigned by devs	Are devs proactively picking up work?	Tracked
Clarification stall rate	Are customers responding to gap questions?	Tracked
LLM cost per customer per week	Is automation within operational budget?	Monitored
Circuit breaker activations	Which customers need KB improvement?	0 in Phase 2+
Playbook library growth rate	Is the system accumulating execution memory?	> 10/week

14. Decision Log

#	Decision	Rationale
D1	Classification always-on, no toggle	Prevents a class of bugs where unclassified tasks break downstream logic
D2	Automation toggle = plan generation only	Separates observability (always) from action (configurable)
D3	Manual trigger for all execution at MVP	Build human confidence before removing human gates
D4	No credential storage in Hub	Products expose provisioned access via MCP/Tools. Hub never sees credential values.
D5	Karpathy LLM Wiki over RAG	Accumulated cross-linked knowledge vs rediscovering from scratch every query
D6	Hub = Intent, Zoho = Execution	Prevents data conflicts. DIRECT_ZOHO_EDIT flag handles exceptions.
D7	Manual re-classification only	Automated scope change detection adds complexity without proportional MVP value
D8	Status via reply thread, not portal	Status sync + LLM contextual reply sufficient for Phase 1
D9	Vercel AI SDK	Aligns with 100% Next.js / Vercel stack. Native MCP, chat, streaming, HITL.
D10	Haiku for frequency, Sonnet for reasoning	Cost and performance optimized per layer. Configurable via DB.
D11	Digest pre-compiled by Cron	Keeps dashboard fast and LLM costs predictable
D12	Requirements assessment = structured DB record	Must be queryable, reportable, and improvable over time
D13	Two separate KB tiers	Different owners, update cadences, access controls. Shared schemas maintained once.
D14	Dev owns code wiki, PM owns documents	Aligns ownership with expertise
D15	Full autonomous execution is Phase 4	Trust earned incrementally through measurable playbook confidence

15. Open Items

#	Item	Owner	Priority
O1	Complete Task Type Taxonomy — full Phase 1 list with min requirements checklists	PM + Dev	■ HIGH
O2	Complete Tenant Configuration Schema — all product-specific IDs per product line	PM + Dev	■ HIGH
O3	Zoho API access and webhook setup (test environment)	Dev	■ HIGH
O4	Sanity API access per tenant — confirm sanity_project_id as isolation key	Dev	■ HIGH

#	Item	Owner	Priority
O5	Claude Code execution environment (GitHub Actions, cloud runtime)	Dev	■ HIGH
O6	Seed playbook content for Content Update and Settings task types	PM + Dev	■ HIGH
O7	Internal KB initial content (product how-tos, shared schemas, workflow docs)	Dev Team	■ HIGH
O8	Vercel AI SDK evaluation and project scaffold	Dev	■ MEDIUM
O9	Confirm model string identifiers for llm_config table	Dev	■ MEDIUM
O10	Developer pool configuration UI for Phase 2 round-robin	PM	■ MEDIUM
O11	Load threshold defaults for round-robin cap	PM	■ MEDIUM
O12	Cliq notification templates and channel structure	PM	■ LOW

End of Document · Version 0.1 Draft · Next review: after PM walkthrough of Section 11 · WebriQ © 2026